

Contents

AI Coding Cost Optimisation	1
Executive summary	1
The problem	1
When not to use AI	1
The maths	2
Token efficiency: the mechanics	2
Open Brain: the memory layer	7
Machine-specific setup	7
The workflow	8

AI Coding Cost Optimisation

A practical guide to cutting AI coding costs by 90% without sacrificing quality.

July 2026

Executive summary

1. **Switch to OpenRouter.** Use free models for 80% of work. Reserve paid models for architecture, security, and production.
2. **Enable prompt caching.** Keep static content at the start of every prompt. Do not rearrange between sessions.
3. **Send only what matters.** Use @file references. Be specific about line numbers. Omit unchanged code.
4. **Batch related work.** One request for three bugs, not three requests for one bug each.
5. **Install Ollama on desktop.** Run Qwen 2.5 Coder 7B locally for simple tasks.
6. **Set up Open Brain.** 45 minutes of setup saves 15-40 minutes per day of context reloading.
7. **Adopt model cascading.** Free first, paid only on failure.

Total monthly cost: €15-20. Previous spending: \$100-300. The tools are free. The discipline is not.

The problem

AI coding assistants burn money fast. At heavy usage rates—roughly 13.5 million tokens per hour—a developer can blow through \$50 worth of credits in under a day. Most of this spending is wasteful: re-loading context that the system already knows, using expensive models for trivial tasks, and failing to exploit free alternatives that have closed the quality gap to within 5-15% of premium offerings.

The solution is not to spend less time with AI. It is to spend smarter.

When not to use AI

AI is not always the right tool. Using it for every task wastes tokens and time.

Skip AI for: - One-line config changes, typo fixes, or simple variable renames. Do it yourself in seconds. - Tasks you already know how to do. Writing a bash command from memory is faster than prompting an AI to write it. - Debugging when you have not read the error message. Read the stack trace first. The answer is often staring at you. - Exploratory work where you need to understand the system. Reading code yourself builds understanding that delegating to AI does not.

Use AI for: - Multi-file changes where pattern consistency matters. - Boilerplate generation (tests, types, API clients). - Code review and second opinions on architecture. - Tasks outside your primary language or framework.

The cheapest token is the one you never send.

The maths

Free models on OpenRouter now handle 80% of coding tasks competently. NVIDIA's Nemotron 3 Ultra, Google's Gemma 4, and similar offerings cost nothing and return results in 3-5 seconds. They fail on complex architecture decisions and security-critical code—precisely where paid models earn their keep.

The cost structure breaks down as follows:

Strategy	Monthly cost	Hours of use	Quality
MiMo Code (\$50/38B credits)	\$50	~22 hours	High
OpenRouter free + \$20 buffer	\$20	Unlimited free + paid	High
Local Ollama (2080 Ti)	~€13 electricity	Unlimited	Good

The free-plus-buffer approach delivers 46% more usage per dollar than MiMo Code, with no sacrifice in output quality for routine work.

Token efficiency: the mechanics

Token efficiency is not a single technique. It is a set of habits—avoiding unnecessary tokens, caching what you can, and choosing the right model for the task—that, taken together, reduce cost by 90-95%.

1. Prompt caching

Most providers (OpenRouter, OpenAI, DeepSeek) cache the beginning of a prompt automatically. If your system instructions, project conventions, and file context appear first, they are served from cache on subsequent requests at 90% lower cost.

How to use it: Put static content at the start of every prompt. In OpenCode, this means your `AGENTS.md` and project structure. Do not rearrange context between sessions. The cache key is positional—moving content breaks it.

What gets cached: System prompts, tool definitions, file context that does not change, conversation history.

What does not: New user messages, new file content, anything that changes between requests.

2. Context window management

The biggest waste is sending irrelevant code. Every token costs money. Sending an entire repository when you need one file is like hiring a moving company to carry a book.

Bad: “Here’s my codebase: [paste 50 files]” — 10,000 tokens.

Good: “Fix the bug in `@src/auth/login.ts` line 42. The session cookie is not being validated.” — 2,000 tokens.

How to use it: Before each request, ask: Am I sending only what the model needs? Use `@file` references. Omit unchanged files. Be specific about line numbers and symptoms.

3. Response length control

Models generate until they run out of tokens or hit a limit. Setting `max_tokens` based on task complexity prevents waste.

Task	max_tokens	Rationale
Simple fix	500	Short response
Code review	1000	Medium response
Feature plan	2000	Detailed response
Complex refactor	4000	Full response

In OpenCode, add to `~/.config/opencode/opencode.json`:

```
{
  "agent": {
    "build": {
      "max_tokens": 2000
    }
  }
}
```

4. Model cascading

The principle: try the cheapest model first. Escalate only when quality is insufficient.

1. Free model writes the plan.
2. Different free model reviews it.
3. Free model implements the code.
4. Free model reviews the implementation.
5. Paid model intervenes only on failure.

When to escalate: - Complex architecture → DeepSeek-V4-Flash (\$0.09/M tokens) - Security features → Qwen3-Coder-Flash (\$0.20/M tokens) - Production deployment → MiMo-V2.5-Pro (\$0.43/M tokens)

5. Failure detection

The problem with cascading: AI models hallucinate confidently. A free model might produce plausible-looking but wrong output. You need verification systems, not trust.

Hard signals (unambiguous failure): - Build fails. The code does not compile. - Tests fail. The code compiles but does not work. - Linter catches errors. Style violations, unused imports, type mismatches. - Type checker rejects the code. TypeScript, mypy, or similar catches structural errors.

These are binary. The model failed. Escalate.

Soft signals (suspicious output): - Cross-model review catches issues. A different model flags problems the first model missed. - Self-consistency check. Ask the same question twice. If the answers diverge, the model is uncertain. Do not trust either. - Hallucinated APIs. The model references functions, libraries, or parameters that do not exist. Verify with documentation. - Output does not match spec. Did it actually solve the problem, or did it solve a different problem confidently? - Overly confident tone. Models that say “this is definitely correct” are often wrong. Hedging is more honest.

The verification gate:

The cascade is not “free model produces output, human decides if it looks right.” That fails because humans are bad at detecting confident hallucinations. The cascade is:

1. Free model produces output.
2. Automated verification runs: build, lint, test.
3. If all pass, accept. If any fail, escalate.
4. On escalation, a different free model reviews and attempts to fix.
5. If that also fails verification, escalate to paid model.

Verification is the gate, not confidence. A free model that passes all tests is more trustworthy than a paid model that fails them.

What verification looks like in practice:

Check	Tool	Pass condition	Fail action
Compile	<code>make build</code>	Exit code 0	Escalate
Lint	<code>make lint</code>	Exit code 0	Fix or escalate
Test	<code>make test</code>	All tests pass	Escalate
Type check	<code>tsc --noEmit</code>	Exit code 0	Escalate
Cross-review	Different model	No issues found	Escalate

The `@verify` agent in the OpenCode config already implements this pattern. It runs build/lint/test before approving work. Use it.

When to skip verification: - Exploratory work (prototyping, research) where output is not final. - Documentation where correctness is subjective. - Plans and designs that will be reviewed by humans anyway.

When to always verify: - Code that will be committed or deployed. - Security-sensitive changes. - Database migrations. - API contract changes.

6. Choosing the right verification

Not all tasks need the same level of verification. Applying full build/lint/test to a one-line doc fix wastes time. Skipping verification on a database migration invites disaster. Match the verification depth to the risk.

Decision matrix:

Task type	Risk level	Verification approach	Time cost
Typo fix, comment update	Trivial	Visual scan only	0 seconds
Simple bug fix	Low	Lint + existing tests	10 seconds
New feature (single file)	Medium	Build + lint + targeted tests	30 seconds
New feature (multi-file)	High	Full build + lint + all tests + cross-review	1-2 minutes
Refactor (no behaviour change)	High	Full build + lint + all tests (regression check)	1-2 minutes
Security change	Critical	Full build + lint + all tests + manual review	5+ minutes
Database migration	Critical	Full build + lint + all tests + rollback plan	5+ minutes
API contract change	Critical	Full build + lint + all tests + integration tests	5+ minutes

The rule of thumb: if the change could break production, verify it like it will. If it cannot break anything, skip verification and move on.

Practical implementation:

For trivial changes (typos, comments, docs):

```
# Just commit. No verification needed.
git add . && git commit -m "fix typo"
```

For low-risk changes (single-file bug fix):

```
# Lint + run existing tests
make lint && make test
```

For medium-risk changes (new feature, single file):

```
# Build + lint + write and run tests
make build && make lint && make test
```

For high-risk changes (multi-file, refactor, security):

```
# Full verification + cross-model review
make build && make lint && make test
# Then invoke @pr-review or @phase-review agent
```

Cross-model review as a verification layer:

When automated checks pass but the change is significant, have a different model review the output. This catches semantic errors that compilers and linters miss: logic bugs, architectural mistakes, security vulnerabilities, performance issues.

Review type	When to use	What it catches
@verify agent	Every code change	Build failures, lint errors, test failures
@pr-review agent	Before merging PRs	Code quality, style, logic bugs
@phase-review agent	Before closing phases	Architectural consistency, completeness
@security-review agent	Security-sensitive changes	Vulnerabilities, injection, auth flaws

Escalation path:

1. **Trivial** (typo, doc) → Visual scan → Commit
2. **Low** (simple fix) → Lint + test → Commit
3. **Medium** (new feature) → Build + lint + test → Commit
4. **High** (refactor, security) → Full verify + cross-review → PR

The overhead of full verification on trivial changes is wasted time. The overhead of skipping verification on critical changes is broken production. Calibrate accordingly.

7. Worked example: adding a new API endpoint

To make this concrete, here is the full verification flow for adding a `POST /api/users` endpoint to an Express.js application. The task is high-risk: multi-file change, new database interaction, new API contract.

Step 1: Free model generates the plan. A different free model reviews it and catches a missing rate-limiting middleware that the first model overlooked.

Step 2: Free model implements the code. It writes three files following existing patterns.

Step 3: Automated verification fails. `make build` passes, `make lint` passes, but 3 of 12 tests fail: wrong status codes, missing error handling, and a password hash leak. This is a hard signal — do not commit.

Step 4: Escalate to fix. The model fixes the three failures. Re-run: all 12 tests pass.

Step 5: Cross-model review catches a security issue. The `@pr-review` agent finds that the password hash is returned in the error path for duplicate emails — something the tests missed. Fix and re-verify.

Step 6: Final commit. All checks pass. Commit.

What this flow caught:

Stage	What was caught	Caught by
Plan review	Missing rate limiting	Cross-model review
Implementation	Wrong status codes, missing error handling, password leak	Automated tests

Stage	What was caught	Caught by
Code review	Password hash in error response	Cross-model review (security)

Without verification, all three issues would have reached production. The free model produced code that compiled and looked correct. Only automated testing and cross-model review caught the bugs.

Time cost: ~3 minutes for plan + implementation + verification. A paid model would have taken the same time but cost \$0.50-2.00 instead of \$0.00.

8. Batching

Three separate bug-fix requests burn three sets of context-loading tokens. One request covering all three bugs saves 40-60%.

Bad: Three requests: “Fix login.ts”, “Fix register.ts”, “Fix reset-password.ts”.

Good: One request: “Fix these three bugs: login.ts line 42 (session cookie), register.ts line 15 (missing validation), reset-password.ts line 28 (token expiry).”

9. Local preprocessing

On a desktop with a 2080 Ti, Ollama runs Qwen 2.5 Coder 7B at 30-50 tokens per second. It handles code formatting, simple refactors, and test generation without touching the network.

```
ollama run qwen2.5-coder:7b "Fix the syntax error in this code: [paste code]"
```

Use for: Formatting, simple refactors, bug fixes, test generation. **Do not use for:** Complex architecture, multi-file changes, security decisions.

Open Brain: the memory layer

Context reloading is the single largest source of waste. Every new session re-explains project structure, conventions, and recent decisions. Open Brain—an open-source system that gives every AI tool the same persistent memory via vector search and MCP—solves this. You store decisions, patterns, and context once; every future session starts with the AI already knowing your project.

Setup time: 15-45 minutes depending on cloud or offline deployment. **Detailed guide:** see [open-brain-guide.md](#).

Machine-specific setup

Chromebook

No GPU, no local models. Use OpenRouter exclusively.

1. Install OpenCode: `curl -fsSL https://opencode.ai/install | bash`
2. Connect OpenRouter: `/connect`, select OpenRouter, paste key
3. Set free models as default (already configured in global config)
4. Set up Open Brain for persistent memory

Budget: \$0-5 per day.

Desktop (i9-9900K / 2080 Ti)

Full GPU available. Use local models for most work.

1. Install Ollama: `curl -fsSL https://ollama.ai/install.sh | sh`
2. Pull coding model: `ollama pull qwen2.5-coder:7b`
3. Install OpenCode: `curl -fsSL https://opencode.ai/install | bash`
4. Connect OpenRouter (for fallback): `/connect`, paste key
5. Set up Open Brain for persistent memory

Budget: €13/month electricity plus \$0-10 API.

Dual machine

Use the Chromebook for mobile work with free API models. Use the desktop for offline and privacy-sensitive tasks with local models. Open Brain syncs knowledge between both.

Combined cost: €15-20 per month.

The workflow

Put it all together:

1. **Start of session:** AI searches Open Brain for relevant context. No re-explaining.
2. **Planning:** Free model writes the plan. Different free model reviews it.
3. **Implementation:** Free model writes code. Send only relevant files via `@file` references.
4. **Verification:** Free model runs build/lint/tests. Escalate to paid model only on failure.
5. **Capture:** Store decisions, patterns, and debugging notes in Open Brain for next time.

Per-feature cost drops from \$5-20 (premium models, no caching, no memory) to effectively zero.

Sources: OpenRouter pricing (June 2026), MiMo Code credit documentation, Open Brain repository (github.com/NateBJones-Projects/OB1), independent model benchmarks (LiveCodeBench, Coding Index, Agentic Index). Benchmark figures are approximate and sourced from public leaderboards at time of writing. Costs are in the currency quoted by each provider; EUR 1 is approximately USD 1.10 at time of writing.